Übungsblatt 2 – Zahlendarstellung

Grundlagen Informatik | DHBW

Aufgabe 1 – Grundbegriffe der Gleitkommadarstellung

(a) Erklären Sie die drei Bestandteile einer Gleitkommazahl nach dem Schema $Z = S \cdot M \cdot B^E$: Was bedeuten Vorzeichen S, Mantisse M und Exponent E? Welche Basis B wird in der Computertechnik üblicherweise verwendet?

(b) Was versteht man unter einer **normalisierten Mantisse**? Geben Sie den zulässigen Wertebereich für Basis 2 an.

(c) Was ist das **Hidden Bit**? Warum ist es ein nützlicher Trick und in welchem Standard wird es eingesetzt?

(d) Was ist der Unterschied zwischen dem **Exponenten** E und der **Charakteristik** C (auch *Bias-Exponent*) bei IEEE 754? Geben Sie die Formel für 32-Bit Float und 64-Bit Double an.

(e) Erklären Sie die Regel der **Kommaverschiebung** bei der Gleitkommadarstellung. Was passiert mit dem Exponenten, wenn das Komma nach rechts bzw. nach links verschoben wird? Geben Sie die Zahl 10400 in mindestens fünf verschiedenen Gleitkommadarstellungen der Form $M \cdot 10^E$ an.

Aufgabe 2 – Nachkommastellen: Dezimal $\leftrightarrow$ Binär *(Berechnung)*

(a) Wandeln Sie $0{,}6875_{10}$ in eine Binärzahl um. Verwenden Sie die **Multiplikationsmethode** und zeigen Sie jeden Schritt.

(b) Wandeln Sie $0{,}3_{10}$ in eine Binärzahl um (5 Nachkommastellen genügen). Was fällt Ihnen auf?

(c) Wandeln Sie $101010{,}010101_2$ vollständig in die Dezimaldarstellung um und geben Sie anschließend die Hexadezimaldarstellung an.

(d) Erklären Sie, warum $0{,}2_{10}$ im Dualsystem nicht endlich darstellbar ist. Welche Konsequenz hat das für die Informatik?

Aufgabe 3 – IEEE 754: Kodierung und Dekodierung *(Berechnung)*

(a) Kodieren Sie $-42{,}328125_{10}$ als **32-Bit Float** nach IEEE 754. Führen Sie alle Schritte durch: Vorzeichenbit, Umwandlung in Binär, Normalisierung, Berechnung der Charakteristik, Hidden Bit entfernen, Mantisse auffüllen.

Geben Sie das Ergebnis in der Form:

$$\underbrace{x}_{VZ}\ \underbrace{x\,x\,x\,x\,x\,x\,x\,x}_{Char.}\ \underbrace{x\,x\,x\ \ldots}_{Mantisse}$$

(b) Dekodieren Sie das folgende 32-Bit-Bitmuster zurück in eine Dezimalzahl:

$$0 \mid 10000010 \mid 10100000000000000000000$$

Führen Sie alle Schritte durch: Charakteristik, Exponent, Mantisse mit Hidden Bit, Endwert.

Aufgabe 4 – Genauigkeit und Grenzen der Fließkommadarstellung

Ein Programmierer schreibt folgenden Pseudocode:

```
a = 1000000000.0
b = 1.0
c = a + b
print(c == a)
```

(a) Welches Ergebnis gibt `print` aus und warum? Erklären Sie den Mechanismus anhand der Denormalisierung bei der Addition.

(b) Zeigen Sie, dass das **Assoziativgesetz** bei Fließkommazahlen verletzt sein kann. Konstruieren Sie ein konkretes Beispiel mit Überlauffalle, ähnlich zu $(10^{60} \cdot 10^{60}) \cdot 10^{-60}$ vs. $10^{60} \cdot (10^{60} \cdot 10^{-60})$.

(c) Erklären Sie anhand eines Beispiels, warum der Präzisionsverlust bei der Addition von Fließkommazahlen unterschiedlicher Größenordnung auftritt. Welche Rolle spielt dabei die begrenzte Mantissenlänge?

Aufgabe 5 – Fließkomma-Arithmetik *(Berechnung)*

Gegeben seien zwei Zahlen in normalisierter Gleitkommadarstellung (Basis 2):

$$Z_1 = 1{,}0110 \times 2^3 \qquad Z_2 = 1{,}1010 \times 2^3$$

(a) **Multiplikation:** Berechnen Sie $Z_1 \cdot Z_2$. Berechnen Sie Mantissenprodukt und Exponentensumme. Normalisieren Sie das Ergebnis und runden Sie auf 4 Nachkommastellen der Mantisse.

(b) **Addition:** Berechnen Sie $Z_1 + Z_2$. Da beide Exponenten gleich sind, ist keine Denormalisierung nötig. Addieren Sie die Mantissen und normalisieren Sie das Ergebnis.

(c) Warum entsteht bei der Multiplikation ein Problem mit der Bitbreite der Mantisse? Was passiert bei 32-Bit-Floats (23-Bit-Mantisse)?

Aufgabe 6 – Wahr oder Falsch

Beurteilen Sie die folgenden Aussagen als **wahr** oder **falsch** und begründen Sie jeweils kurz:

(a) Bei IEEE 754 (32-Bit) ist das Hidden Bit immer 0 und wird weggelassen.

(b) Wenn alle 8 Charakteristik-Bits den Wert 1 haben und alle Mantissenbits 0 sind, repräsentiert das Bitmuster $\pm\infty$.

(c) Die Zahl $0{,}1_{10}$ lässt sich als endlicher Binärbruch darstellen.

(d) Das Assoziativgesetz $a + (b + c) = (a + b) + c$ gilt für IEEE-754-Fließkommazahlen immer.

(e) Bei der Addition zweier Fließkommazahlen mit unterschiedlichen Exponenten muss die kleinere Zahl denormalisiert werden.

(f) Alle Bits 0 ergibt bei IEEE 754 den Wert 0_{10}; alle Charakteristik-Bits 1 und Mantisse 0 ergibt $\pm\infty$.

Lösungen – Übungsblatt 2

Aufgabe 1 – Grundbegriffe der Gleitkommadarstellung

(a) Die allgemeine Form einer Gleitkommazahl lautet $Z = S \cdot M \cdot B^E$:

- **Vorzeichen** S: nimmt den Wert $+1$ oder -1 an und bestimmt, ob die Zahl positiv oder negativ ist. In IEEE 754 wird es als ein einzelnes Bit kodiert (0 = positiv, 1 = negativ).
- **Mantisse** M (auch *Signifikand*): enthält die signifikanten Stellen der Zahl und bestimmt die Genauigkeit. Bei Basis 2 enthält sie die Vorkomma-1 und alle Nachkommabits.
- **Exponent** E: gibt an, um wie viele Stellen das Komma verschoben wird. Ein positiver Exponent verschiebt nach rechts (größere Zahl), ein negativer nach links (kleinere Zahl).

In der Computertechnik wird fast ausnahmslos Basis $B = 2$ verwendet, da alle Berechnungen auf binärer Hardware stattfinden.

(b) Eine normalisierte Mantisse erfüllt die Bedingung $1 \leq M < B$. Bei Basis 2 bedeutet das: $1{,}000\ldots \leq M < 1{,}111\ldots$, also hat die Mantisse immer genau eine führende 1 vor dem Komma. Dies ist die kanonische Form und vermeidet mehrdeutige Darstellungen derselben Zahl (z. B. $1{,}01 \times 2^3$ und $10{,}1 \times 2^2$ wären sonst äquivalent).

(c) Das **Hidden Bit** nutzt die Tatsache, dass eine normalisierte Mantisse in Basis 2 stets mit einer führenden 1 beginnt. Da diese 1 nie fehlt, muss sie nicht explizit gespeichert werden – sie ist *implizit* vorhanden. IEEE 754 lässt daher die führende 1 weg und gewinnt dadurch ein zusätzliches Präzisionsbit kostenlos. Beim Dekodieren wird die 1 wieder vor die gespeicherten Mantissenbits gesetzt: $1{,}\langle$gespeicherte Bits$\rangle$.

(d) Der Exponent E ist der mathematische (biasfreie) Exponent, der negativ sein kann. Die Charakteristik C ist eine vorzeichenlose Ganzzahl, die durch Addition eines festen Bias entsteht:

$$C = E + \text{Bias}$$

Durch den Bias können Exponenten ohne separates Vorzeichen gespeichert werden, was den Vergleich von Fließkommazahlen vereinfacht:

- 32-Bit Float: Bias $= 127$, also $C = E + 127$
- 64-Bit Double: Bias $= 1023$, also $C = E + 1023$

(e) Bei der Gleitkommadarstellung $M \cdot B^E$ kann das Komma in der Mantisse beliebig verschoben werden, solange der Exponent entsprechend angepasst wird. Als Eselsbrücke gilt: *dexter* (lateinisch für *rechts*) = *dekrementiert* – wird das Komma nach rechts verschoben, wird der Exponent um eins verringert. Umgekehrt wird der Exponent erhöht, wenn das Komma nach links verschoben wird, da die Mantisse kleiner wird und der größere Exponent dies ausgleichen muss.

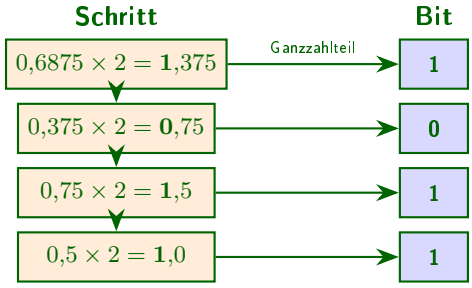
Fünf verschiedene Darstellungen von 10400:

$$10{,}4 \cdot 10^3 \;=\; 0{,}104 \cdot 10^5 \;=\; 104{,}0 \cdot 10^2 \;=\; 0{,}00104 \cdot 10^7 \;=\; 1{,}04 \cdot 10^4$$

Die **normalisierte Mantisse** liegt per Konvention im Bereich $1 \leq M < B$. Bei Basis 10 bedeutet das: genau eine Ziffer ungleich Null vor dem Komma. Von den fünf Darstellungen erfüllt nur $1{,}04 \cdot 10^4$ diese Bedingung – sie ist somit die normalisierte Form. Die Normalisierung sorgt für eine eindeutige Darstellung und wird beispielsweise von IEEE 754 (dort mit Basis 2) zwingend vorgeschrieben.

Aufgabe 2 – Nachkommastellen: Dezimal $\leftrightarrow$ Binär

(a) Multiplikationsmethode für $0{,}6875_{10}$: Man multipliziert wiederholt mit 2 und liest den Ganzzahlteil ab. Das Verfahren endet, sobald der Nachkommateil 0 wird.



Ergebnis: $0{,}6875_{10} = 0{,}1011_2$

(b) Multiplikationsmethode für $0{,}3_{10}$ (5 Stellen):

$$\begin{aligned}
0{,}3 \times 2 &= \mathbf{0}{,}6 &\Rightarrow\quad & 0 \\
0{,}6 \times 2 &= \mathbf{1}{,}2 &\Rightarrow\quad & 1 \\
0{,}2 \times 2 &= \mathbf{0}{,}4 &\Rightarrow\quad & 0 \\
0{,}4 \times 2 &= \mathbf{0}{,}8 &\Rightarrow\quad & 0 \\
0{,}8 \times 2 &= \mathbf{1}{,}6 &\Rightarrow\quad & 1 \\
0{,}6 \times 2 &= \mathbf{1}{,}2 &\Rightarrow\quad & 1 \quad \leftarrow \text{Wiederholung!}
\end{aligned}$$

Ergebnis: $0{,}3_{10} \approx 0{,}01001\overline{1001}_2$ (periodisch). Der Nachkommateil wiederholt sich – $0{,}3_{10}$ ist kein endlicher Binärbruch.

(c) Umrechnung $101010{,}010101_2$:

$$\begin{aligned}
&1{\cdot}2^5 + 0{\cdot}2^4 + 1{\cdot}2^3 + 0{\cdot}2^2 + 1{\cdot}2^1 + 0{\cdot}2^0 \\
&\quad + 0{\cdot}2^{-1} + 1{\cdot}2^{-2} + 0{\cdot}2^{-3} + 1{\cdot}2^{-4} + 0{\cdot}2^{-5} + 1{\cdot}2^{-6} \\
&= 32 + 8 + 2 + 0{,}25 + 0{,}0625 + 0{,}015625 \\
&= 42{,}328125_{10}
\end{aligned}$$

Hexadezimaldarstellung: Die 6 Vorkomma-Bits $101010_2 = 2A_{16}$, die 6 Nachkomma-Bits $010101_2 = 54_{16}$ (je als 4-Bit-Gruppe: 0010 1010,0101 0100). Ergebnis: $\texttt{2A}{,}54_{16}$.

(d) $0{,}2_{10}$ hat im Dualsystem einen periodischen Binärbruch ($0{,}\overline{0011}_2$), weil 5 kein Teiler einer Zweierpotenz ist: $0{,}2 = 1/5$, und 5 ist teilerfremd zu 2^k für alle k. In der Software bedeutet das, dass Literale wie `0.1` oder `0.2` niemals exakt gespeichert werden. Vergleiche wie `0.1 + 0.2 == 0.3` liefern `False`.

Aufgabe 3 – IEEE 754: Kodierung und Dekodierung

(a) Kodierung von $-42{,}328125_{10}$ als 32-Bit Float:

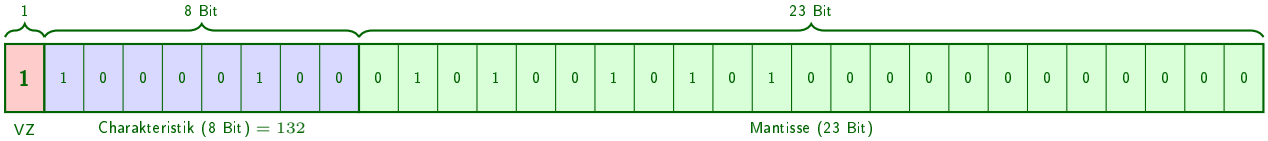
Schritt 1 – Vorzeichen: $S = 1$ (negativ).

Schritt 2 – Umwandlung: $42_{10} = 101010_2$, $0{,}328125_{10} = 0{,}010101_2$, also $42{,}328125_{10} = 101010{,}010101_2$.

Schritt 3 – Normalisierung: $101010{,}010101_2 = 1{,}0101001\,0101_2 \times 2^5$. Exponent $E = 5$.

Schritt 4 – Charakteristik: $C = E + 127 = 5 + 127 = 132 = 10000100_2$.

Schritt 5 – Mantisse (Hidden Bit entfernen, 23 Bit): Mantissenbits nach dem Komma: $01010010101\underbrace{0000000000000}_{\text{auffüllen}} \rightarrow$ 01010010101000000000000



(b) Dekodierung des Bitmusters $0 \mid 10000010 \mid 10100000000000000000000$:

Vorzeichen: Bit $= 0 \Rightarrow$ positiv.

Charakteristik: $10000010_2 = 130_{10}$.

Exponent: $E = C - 127 = 130 - 127 = 3$.

Mantisse mit Hidden Bit: $1{,}101_2$ (Hidden Bit 1 davor, dann $101\underbrace{00\ldots}_{20}$).

Endwert:

$$Z = 1{,}101_2 \times 2^3 = 1101_2 = 13_{10}$$

Das Bitmuster repräsentiert $+13{,}0_{10}$.

Aufgabe 4 – Genauigkeit und Grenzen der Fließkommadarstellung

(a) `print` gibt `True` aus – denn `c == a`. Das klingt falsch, aber: $a = 10^9$ benötigt ca. 30 Bit zur Darstellung. Bei der Addition $a + b$ muss $b = 1$ auf den Exponenten von a denormalisiert werden: $1{,}0 = 0{,}000000001\ldots \times 2^{30}$. Die Mantisse des 32-Bit-Floats hat nur 23 Bit – die kleine Zahl 1 *fällt aus der Mantisse heraus* und das Ergebnis ist identisch mit a. Dieses Phänomen heißt **Präzisionsverlust durch Auslöschung**.

(b) Beispiel für Assoziativgesetzverletzung:

$$\begin{aligned}
&(10^{60} \cdot 10^{60}) \cdot 10^{-60} \\
&= 10^{120} \cdot 10^{-60} = 10^{60} \quad \checkmark \text{ (Überlauf bei Zwischenergebnis!)} \\
&10^{60} \cdot (10^{60} \cdot 10^{-60}) \\
&= 10^{60} \cdot 10^{0} = 10^{60} \quad \checkmark
\end{aligned}$$

Im ersten Fall überläuft 10^{120} den darstellbaren Bereich (∞), im zweiten nicht. Das Assoziativgesetz gilt also *nicht* in Fließkomma-Arithmetik.

(c) Der Präzisionsverlust entsteht durch die begrenzte Mantissenlänge von 23 Bit bei 32-Bit-Floats. Bei der Addition zweier Zahlen mit unterschiedlichen Exponenten muss die kleinere Zahl **denormalisiert** werden: ihre Mantisse wird nach rechts geschoben, bis beide Exponenten übereinstimmen. Dabei fallen Bits am rechten Ende der Mantisse heraus.

Beispiel: $a = 10^{10}$, $b = 1$. Bei der Berechnung von $a + b$ muss $b = 1{,}0$ auf den Exponenten von a denormalisiert werden. Dabei wird die Mantisse von b so weit nach rechts verschoben, dass sie vollständig aus der 23-Bit-Mantisse herausfällt. Das Ergebnis ist $a + b = a$ – die kleinere Zahl geht komplett verloren. Je größer der Exponentenunterschied, desto mehr Bits der kleineren Zahl gehen bei der Denormalisierung verloren.

Aufgabe 5 – Fließkomma-Arithmetik

(a) **Multiplikation** $Z_1 \cdot Z_2$:

$$\begin{aligned} M_1 \cdot M_2 &= 1{,}0110_2 \times 1{,}1010_2 \\ &= 10{,}00111100_2 \quad \text{(8-Bit-Produkt)} \\ E_1 + E_2 &= 3 + 3 = 6 \end{aligned}$$

Normalisierung: $10{,}00111100_2 \times 2^6 = 1{,}000111100_2 \times 2^7$. Auf 4 Mantissennachkommabits gerundet: $1{,}0010_2 \times 2^7$.

Ergebnis: $Z_1 \cdot Z_2 \approx 1{,}0010_2 \times 2^7 = 10010000_2 = 144{,}0_{10}$ (exakt wäre $1{,}0110 \times 1{,}1010 \times 2^6 = 11 \times 13 = 143$ – die gerundete Darstellung weicht ab).

(b) **Addition** $Z_1 + Z_2$ (gleicher Exponent 2^3):

$$\begin{aligned} M_1 + M_2 &= 1{,}0110_2 + 1{,}1010_2 \\ &= 11{,}0000_2 \end{aligned}$$

Normalisierung: $11{,}0000_2 \times 2^3 = 1{,}1000_2 \times 2^4$.

Ergebnis: $Z_1 + Z_2 = 1{,}1000_2 \times 2^4 = 11000_2 = 24_{10}$.

(c) Bei der Multiplikation zweier Mantissen der Breite n entsteht ein Produkt der Breite $2n$. Bei 32-Bit-Floats mit 23-Bit-Mantisse ergibt $23 \times 23 = 46$ Bit. Diese 46 Bit passen nicht in die 23-Bit-Mantisse, daher muss auf 23 Bit gerundet werden. Jede Multiplikation führt so zwingend zu einem Rundungsfehler, der sich bei wiederholten Operationen akkumulieren kann.

Aufgabe 6 – Wahr oder Falsch

(a) **Falsch.** Das Hidden Bit ist immer **1**, nicht 0. Bei normalisierten Zahlen beginnt die Mantisse in Basis 2 stets mit einer führenden 1, die weggelassen (gespart) wird. Eine gespeicherte Mantisse 10000... bedeutet also $1{,}1_2$, nicht $0{,}1_2$.

(b) **Wahr.** Wenn alle 8 Charakteristik-Bits gesetzt sind (Wert 255 bei 32-Bit) und alle Mantissenbits 0 sind, repräsentiert das Bitmuster $\pm\infty$ (Vorzeichen aus dem VZ-Bit). Dies ist einer der beiden Sonderwerte im IEEE-754-Standard: "Alle Bits = 0" ergibt die Zahl 0, und "alle Charakteristik-Bits = 1 bei Mantisse 0" ergibt $\pm\infty$. Die Aussage ist wahr, da sie genau diesen Sonderfall beschreibt.

(c) **Falsch.** $0{,}1_{10} = 0{,}0\overline{0011}_2$ ist ein periodischer, nicht endlicher Binärbruch. Kein endliches Bitmuster kann $0{,}1$ exakt darstellen – es wird gerundet gespeichert.

(d) **Falsch.** Das Assoziativgesetz gilt bei Fließkommazahlen *nicht*, weil die Reihenfolge der Operationen zu unterschiedlichen Rundungsfehlern führt. Beispiel aus dem Lernzettel: $(10^{60} \cdot 10^{60}) \cdot 10^{-60}$ ergibt einen Überlauf (10^{120}), während $10^{60} \cdot (10^{60} \cdot 10^{-60}) = 10^{60}$ korrekt berechnet wird.

(e) **Wahr.** Bei der Addition von $Z_1 + Z_2$ mit $E_1 \neq E_2$ muss die Zahl mit kleinerem Exponenten so verschoben werden, dass beide Exponenten gleich sind. Dabei wird die Mantisse nach rechts verschoben (Denormalisierung), bevor die Addition durchgeführt wird.

(f) **Wahr.** Der Sonderfall "alle Bits 0" ergibt $\pm 0_{10}$ (je nach Vorzeichen-Bit). Der Sonderfall "alle 8 Charakteristik-Bits 1 und Mantisse $= 0$" ergibt $\pm\infty$. Diese Sonderwerte sind fest im IEEE-754-Standard definiert und werden von jeder konformen Hardware korrekt behandelt.